Checkpoint Artifact Choice 1

Catalog item: System Designs — Visual System Design Artifacts
Project: Material Donor Mutual Assist (material-derailleur) BWorks donation management system
Target Sprint: 3

Artifact identification and rationale

I selected the **Visual System Design Artifacts** checkpoint because it directly supports our final product and community strategy. Material-Derailleur is a full-stack application (React, Express, PostgreSQL) with multiple user roles (Admin, Donor, tiered staff), external integrations (Azure Blob Storage, Google GenAI, SMTP), and non- trivial data and workflow. Having a single set of up-to-date visual design artifacts helps the team and future maintainers understand system structure, API boundaries, data model, deployment, and user flows without reading the entire codebase. These artifacts guide current development (e.g. adding endpoints or roles), onboarding, and documentation for open-source contributors, aligning with our community strategy that we are going to create.

What we created and why

We created **five visual design artifacts** for Material-Derailleur:

1. **System & API architecture:** Component diagram of the Express backend: REST route groups, services (donor, program, donatedItem, email, image analysis(still in progress)), and their connections to PostgreSQL (Prisma), Azure Blob Storage, SMTP, and Google GenAI. We chose this to clarify API boundaries and where integration and business logic live.
2. **Data model / entity relationship:** ER diagram derived from server/prisma/schema.prisma (Donor, Program, DonatedItem, DonatedItemStatus, User). This supports database discussions, migrations, and onboarding.
3. **Deployment architecture:** Diagram of Docker Compose services (client-app which is our frontend, server which is out backend, database), ports, and optional external services. It shows how the system is hosted and scaled in our current setup.
4. **User flows** — Donor flow (login --- profile --- my donations --- item details / barcode) and Admin flow (donor/donation management, search, import, status updates, user management, image review). These align with our feature set and help prioritize UX and permissions.
5. **Frontend component & route map** — High level React components and routes (public, donor, admin, programs). This illustrates separation of concerns and where to add new screens or protected routes.

Key decisions and how artifacts inform development

- **Single backend, multiple route modules:** The API diagram makes explicit that auth, donors, programs, donated items, status, barcode, and password reset are separate route groups backed by shared services and Prisma. New features (e.g. reporting) can be placed consistently.

- **User vs Donor:** The ER diagram clarifies authentication, and roles live in User while donation profiles and history live in Donor; this avoids confusion when implementing role-based UI (e.g. ProtectedRoute).
- **Deployment:** The deployment diagram documents that the backend is exposed on port 5050 (mapped from 5000) and depends on a healthy database; this guides environment and CI/CD setup.
- **User flows:** Donor and Admin flows tie features to personas and help ensure donor-facing "donation journey" and admin "check-in and import" workflows are well supported.

Current vs intended state (Existing product)

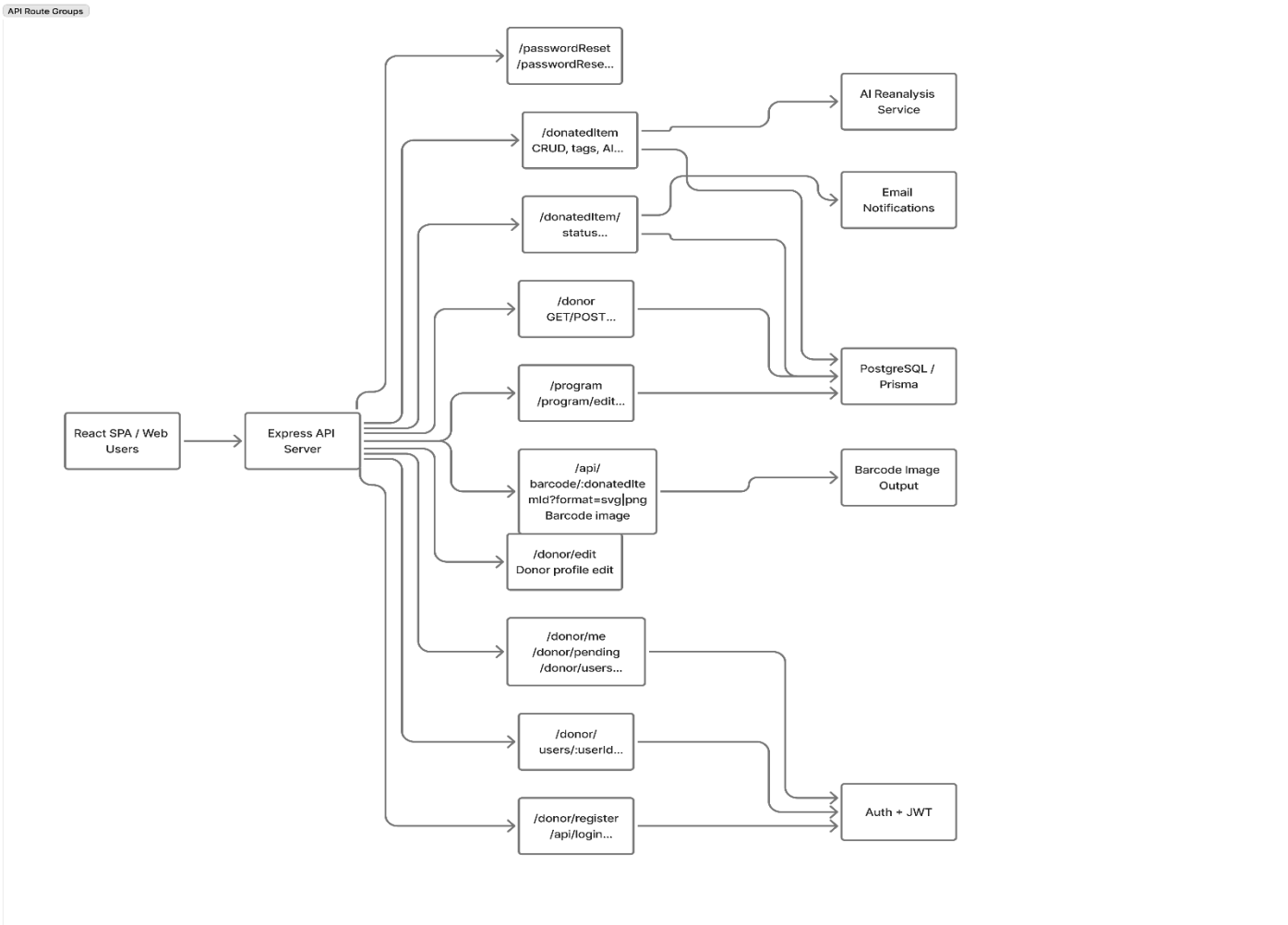
The artifacts describe the **current state** of Material-Derailleur as implemented (routes, Prisma schema, Docker Compose, React routes and main components). Intended future improvements that could be reflected in later iterations include Open API/Swagger for the API, clearer API versioning, and expanded donor-facing notifications, searching donations by barcode, easy of uploading images and an inventory of the donations; the same diagram set can be updated to show those changes when we implement them. The primary objective is to tell a story, but we are going to scale up so that after telling the story we can do the administration of the work Bworks is doing

---

Visual design artifacts

1. System & API architecture

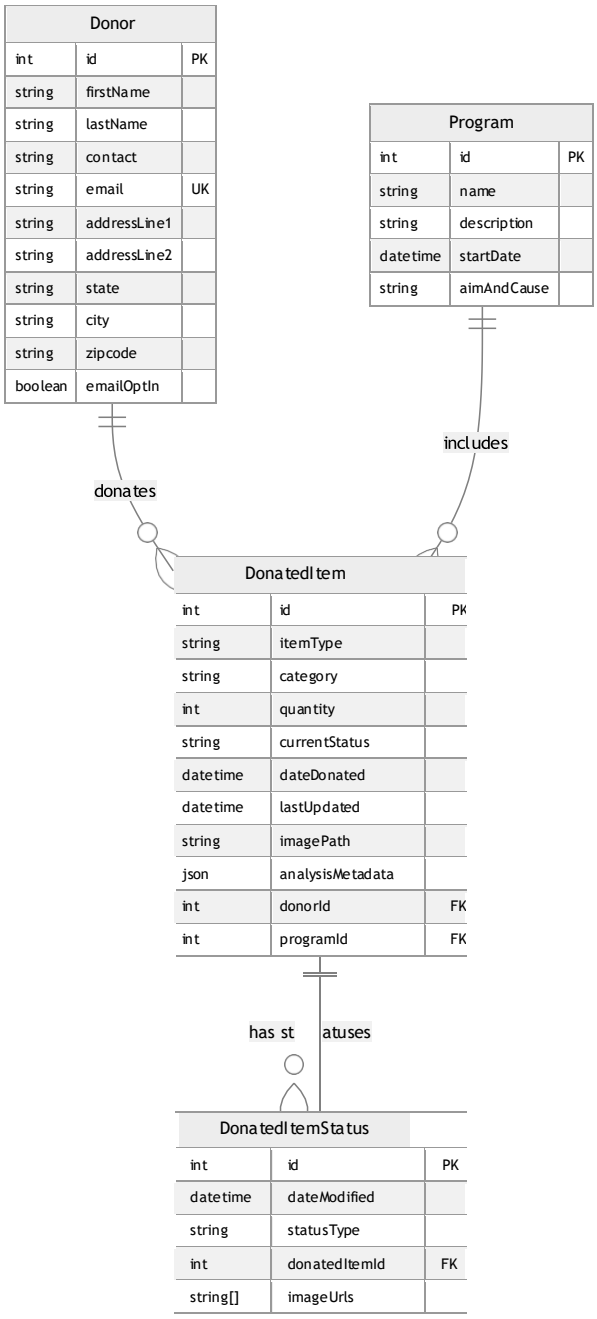
Major system parts, backend services, and their interactions.



Route prefix	Purpose
GET/POST /donor	Donor CRUD, list, search
POST /donor/register, POST /api/login	Auth (register/login)
GET /donor/me, /donor/pending, /donor/users	Current user, pending users, admin user list
PUT /donor/users/:userId	Admin user approval/update
POST /donor/edit	Donor profile edit
POST /passwordReset, /passwordReset/reset-password	Password reset flow
GET/POST/PUT/DELETE /donatedItem, tags, reanalyze	Donated items CRUD, tags, AI reanalysis
POST /donatedItem/status	Add status to donated item
GET/POST /program, POST /program/edit	Programs CRUD
GET /api/barcode/:donatedItemId?format=svg|png	Barcode image

2. Data model & entity relationship

Entity relationship diagram from server/prisma/schema.prisma.

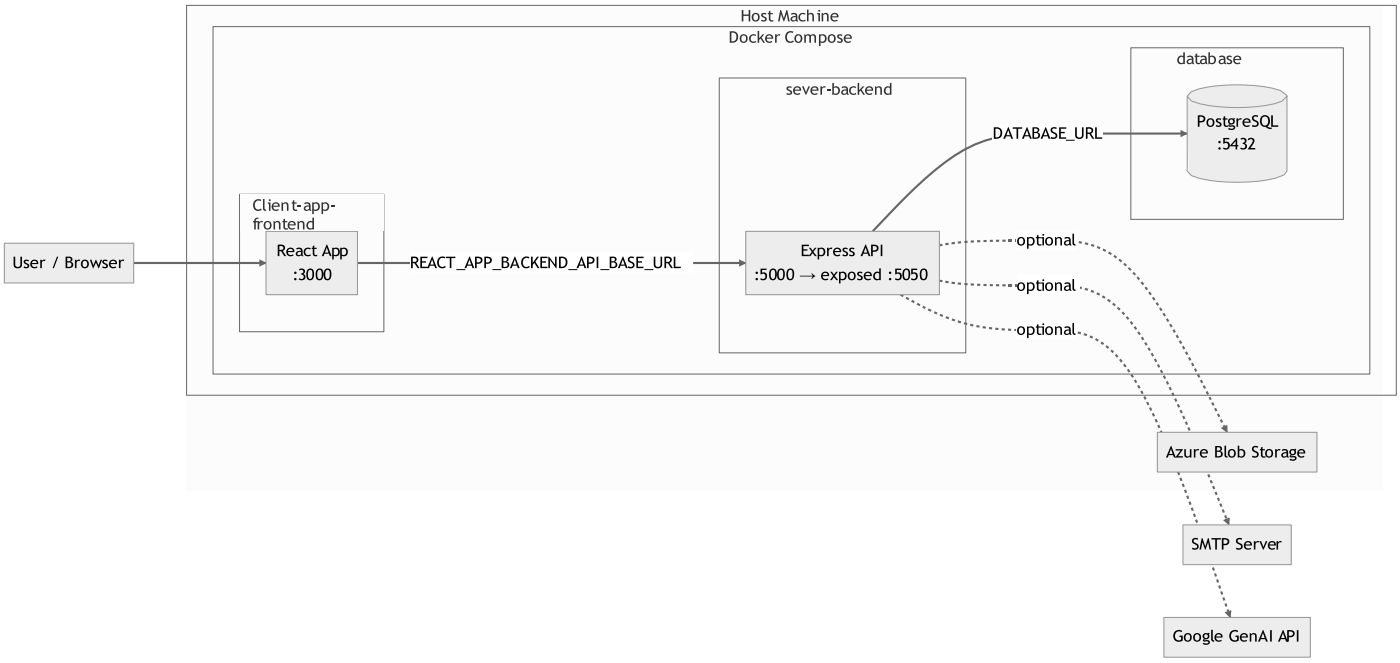


User			
uuid	id	PK	
string	name		
string	email	UK	
string	password		
enum	role		ADMIN,DONOR,TIER_ONE,TIER_TWO,TIER_THREE
enum	status		PENDING,ACTIVE,SUSPENDED
datetime	createdAt		
boolean	firstLogin		
string	resetToken		
datetime	resetTokenExpiry		

Notes: User is separate from Donor (auth/roles in User; donor profile/donations in Donor). DonatedItem.analysisMetadata stores AI (Google Gemini) results. DonatedItemStatus is history; DonatedItem.currentStatus is current value.

3. Deployment architecture

Docker Compose: how the system is hosted and scaled.

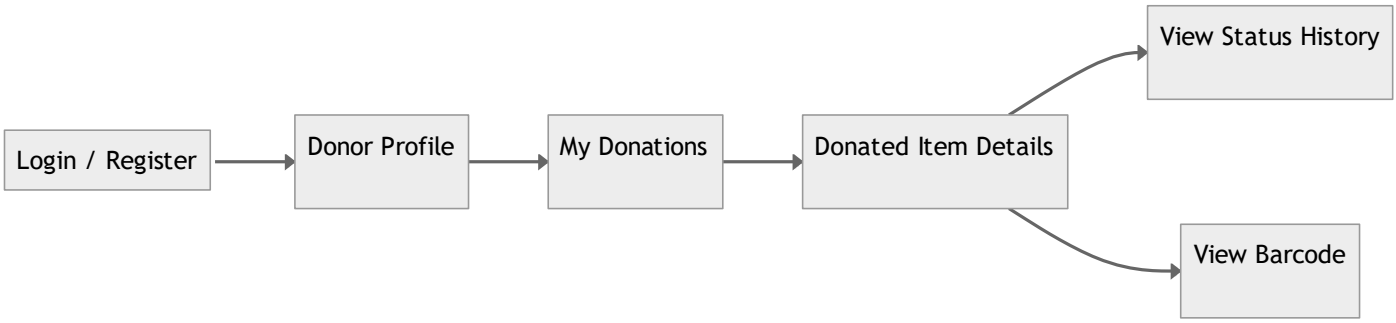


Service	Port (host:container)	Depends on
Client-app-frontend	3000:3000	backend
Server-backend	5050:5000	database (healthy)
database	5432:5432	

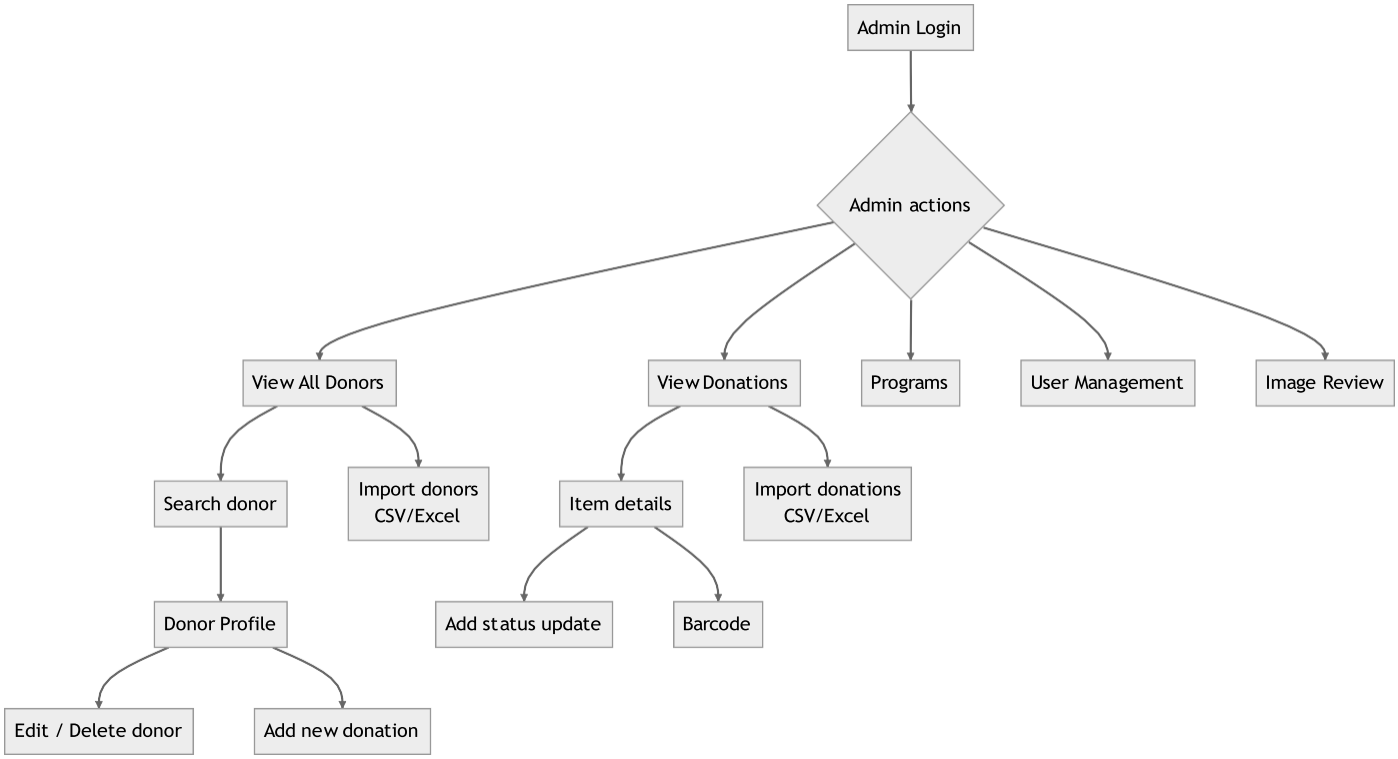
Configuration is driven by .env (e.g. DATABASE_URL, JWT_SECRET, AZURE_*, SMTP_*).

4. User flows

Donor flow: View my donations and their journey



Admin flow: Check-in and manage donors/donations

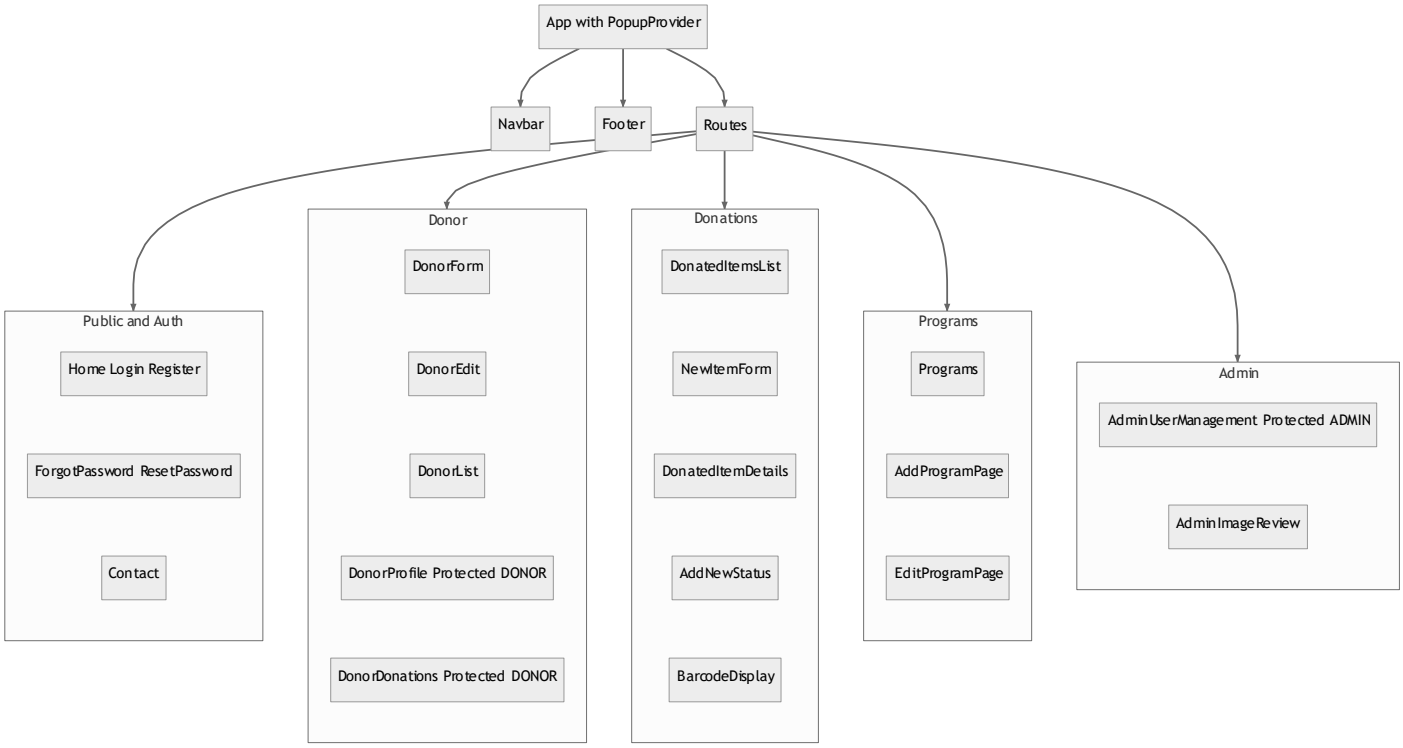


Key screens (information architecture)

Area	Routes	Purpose
Public	/login, /register, /forgot-password, /reset- password, /contact	Landing, auth, contact
Donor	/donor-profile, /my-donations	Donor dashboard and donation list (role-protected)
Admin – Donors	/donorlist, /donorform, /donoredit	Donor CRUD, search, bulk delete, import
Admin – Donations	/donations, /adddonation, /donations/:id, /donatedItem/status/:id	Item list, add, details, status update
Admin – Programs	/programs, /addprogram, /editprogram	Program CRUD
Admin – System	/admin/user-management, /admin/image-review	User approval, image review
Shared	/donations/:id/barcode, /donated/:id/barcode	Barcode display

5. Frontend component & route map

High-level React components and routes (client-app/src/App.js).



Layer	Responsibility
App	Routing, layout (Navbar + main + Footer), auth wrapper (ProtectedRoute)
Auth	LoginPage, Register, ForgotPassword, ResetPasswordPage, ProtectedRoute
Donor	DonorForm, DonorEdit, DonorList, DonorProfile, DonorDonations
Donations	DonatedItemsList, NewItemForm, DonatedItemDetails, AddNewStatus, BarcodeDisplay
Programs	Programs, AddProgramPage, EditProgramPage
Admin	AdminUserManagement, AdminImageReview